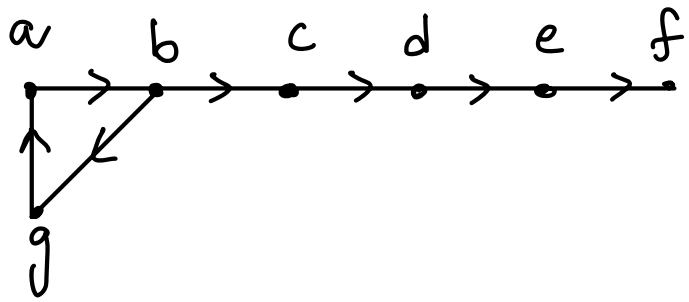
Def:

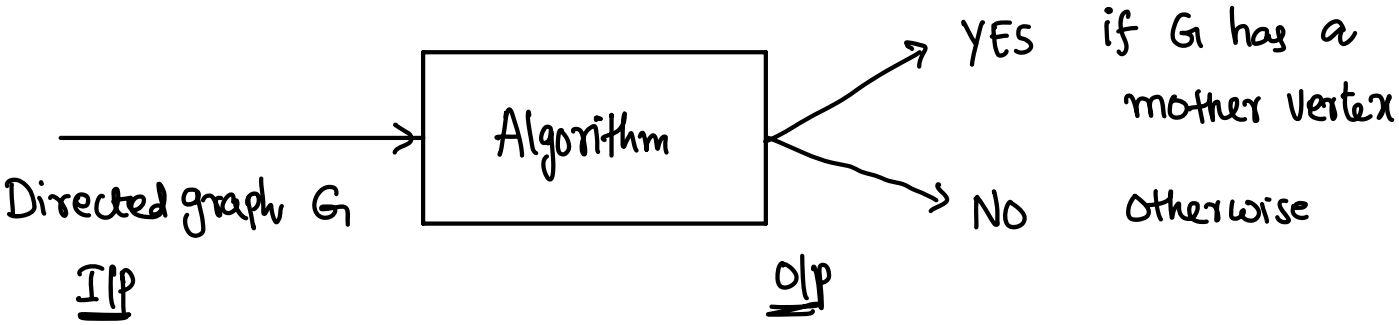
A mother vertex in a graph G is a vertex v such that all other vertices in G are reached by a path from v.

Ex:



both a & g are mother vertices.

Design an algorithm to test whether a graph has a mother vertex.



What is the trivial algorithm ?

TRUE | FALSE :

If a directed graph G has a mother vertex, then in any DFS of G the last finished vertex is a mother vertex.

Proof of True/False :

let z be a mother vertex of G

let v be the vertex with the largest finish time in a DFS of G.

Assume for contradiction that v is not a mother vertex

Then there exists a vertex u such that u is not reachable from v.

Also observe that v can not reach z:

because $v \rightsquigarrow z$ and $z \rightsquigarrow u$

$\Rightarrow v \rightsquigarrow u$ Contradiction to $v \not\rightsquigarrow u$.

Consider the DFS discovery order of v and z

Since v cannot reach z, DFS starting from v cannot discover z.

Thus there are two possibilities

1. z is discovered before v

When DFS explores z, since z is a mother vertex it can reach v.

Thus DFS will discover v while exploring z

Therefore v is a descendant of z in the DFS tree which implies $z.f > v.f$, which is a contradiction to the assumption that v has the largest finishing time.

2. z is discovered after v finishes

Then $z.f > v.f$
again a contradiction

Thus our assumption that v is not a mother vertex is false.

Therefore v must be a mother vertex.

What is the algorithm ?

Algorithm – Mother Vertex

① Run DFS on the whole graph

② Keep track of the last finished Vertex say z

③ Run DFS again starting from the Vertex z

④ If all Vertices are reachable then z is a mother vertex

Otherwise — No mother Vertex exists

Running time: $O(V+E)$

Cut Vertex and Cut edges in a graph

Remark: In this topic we only work with Undirected graphs which are Connected

Cut Vertices & Cut-edges

Articulation Points

bridges

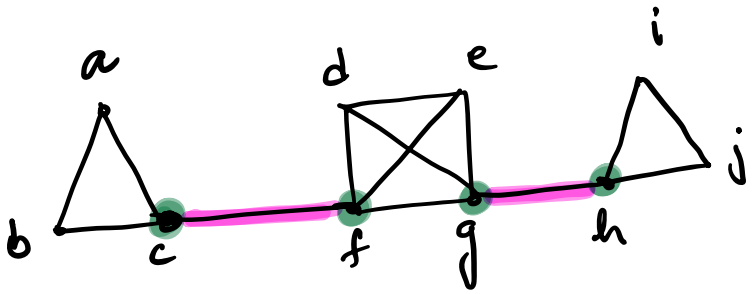
Let $G = (V, E)$ be a **connected**, **undirected** graph.

Def:-

A **cut vertex** of G is a vertex whose removal disconnects G.

A **cut edge** of G is an edge whose removal disconnects G.

Example:



Cut vertices = c, f, g, h

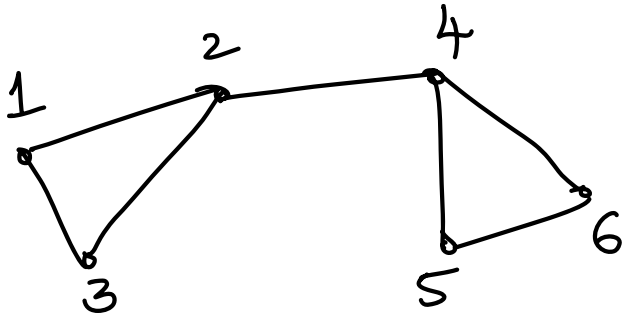
Cut edges = cf, gh

- Cut edges are also called bridges

- Cut Vertices are also called Articulation points

Example:

Find all cut vertices & cut edges in the following graph.



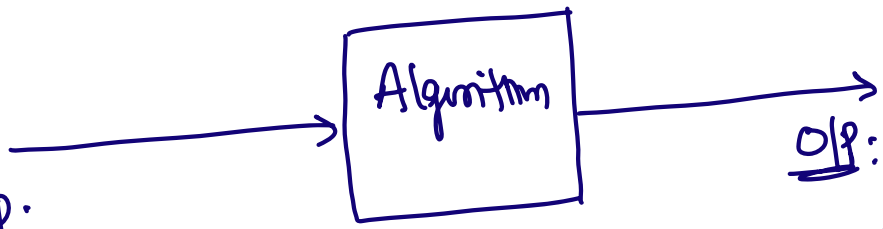
Problem:

Design an algorithm to find

(a) Cut vertices in a graph [Homework]

(b) Cut edges in a graph [Now]

Problem: Finding cut edges (bridges) in a graph



I/P:

Undirected graph G

+

Connected

O/P:

cut edges of G

Easy Algorithms (Warmup)

Brute-force approach

Using BFS

For every edge e

remove e from G

See if the graph remains Connected

if G is not Connected

e is a cut edge

add e back to the graph

Running time: $O(m * (n+m))$

$= O(m^2)$

Q:

Can we do better ?

Q:

Can we do better ?

HINT: ① How can we use DFS

② Remember that we are working with Undirected graphs

Idea:

Since we are working with undirected graphs, DFS search produces tree-edges & back-edges only.

TRUE | FALSE:

A back edge is never a cut-edge.

TRUE | FALSE:

All cut edges must belong to the DFS tree.
(ie., tree-edges)

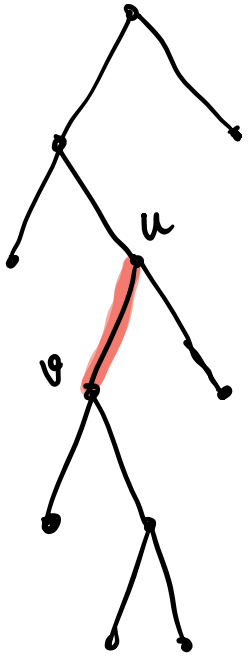
So

we will focus on only tree-edges

T/F: Every tree edge is a cut edge

T/F: Some tree edges may be cutedges

Look at the DFS tree:



Q When uv is NOT a cut edge?

Q When uv is NOT a cut edge?

Ans: If there is a back edge from some decendent of v (including v) to some ancestor (Proper) of v.

(or)

A tree edge uv with u as v's Parent is a cut-edge if and only if there are no edges in v's subtree that goes to u or higher.

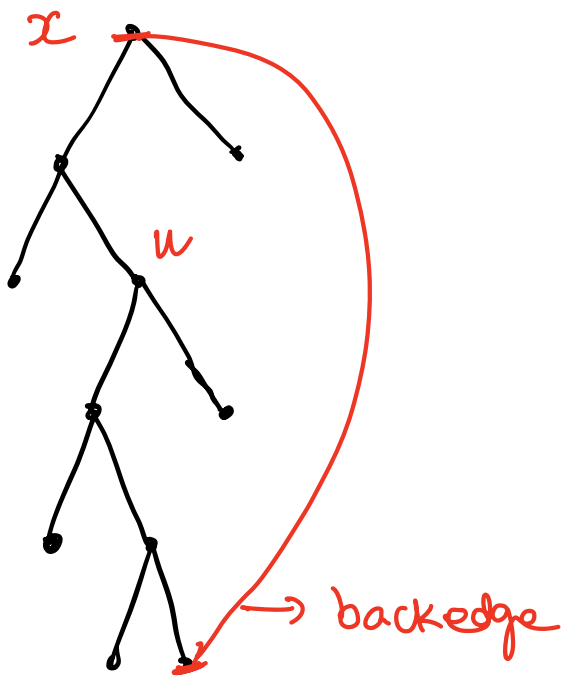
There may be many such back edges

Q Do we need to keep track of all back edges?

Q Do we need to keep track of all back edges?

Ans: For each node u.

We keep track of the highest node x such that there is a back edge from a descendant of u upto x.



Here the "highest" node is the one with the lowest discover time.

$$h[u] = \min\{ x.d \mid w \text{ descendant of } u \text{ and exists a backedge from } w \text{ up to } x \}$$

Note: w may be u also

We set $h[u] = \infty$ if there is no backedge from any descendant of u.

For our algorithm [in implementation]

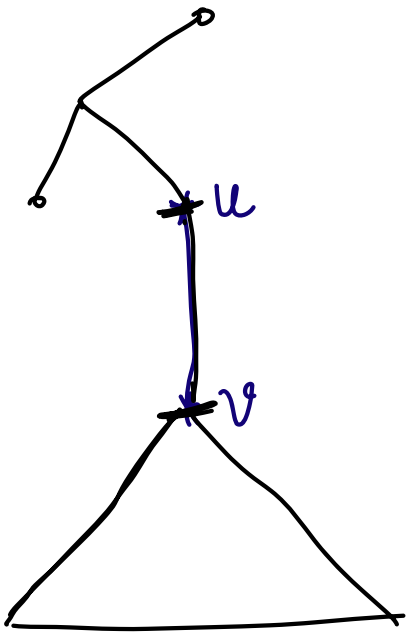
we set $h[u] = u.d$

Now, we need to answer the following.

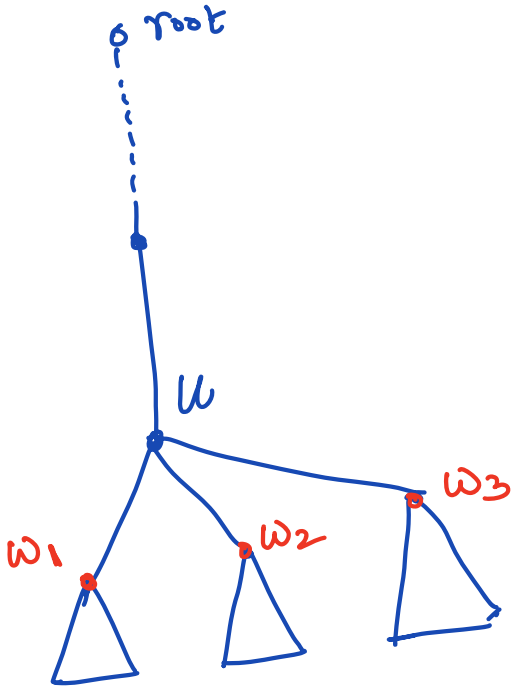
① How to compute $h[u]$ for each node u?

② How can we use $h[u]$ values to compute cut-edges?

Q: If a tree edge uv is a cut-edge then what is the relation b/w $h[v]$ and $u.d$?

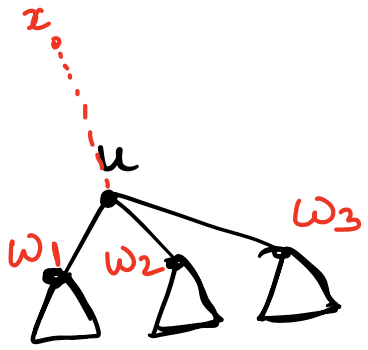


Computing $h[u]$

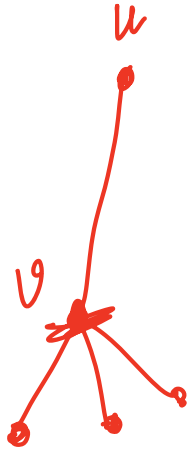


Q: How to compute $h[u]$?

We can define h recursively as follows.



$$h[u] = \min\Big(\{ h[w] \mid w \text{ is a child of } u \text{ in DFS tree} \} \cup \{ x.d \mid \text{exists back edge from } u \text{ up to } x \} \Big)$$

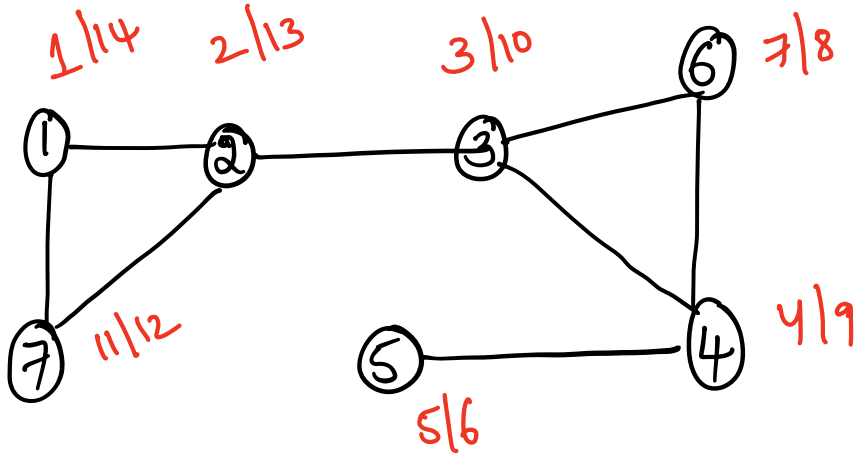


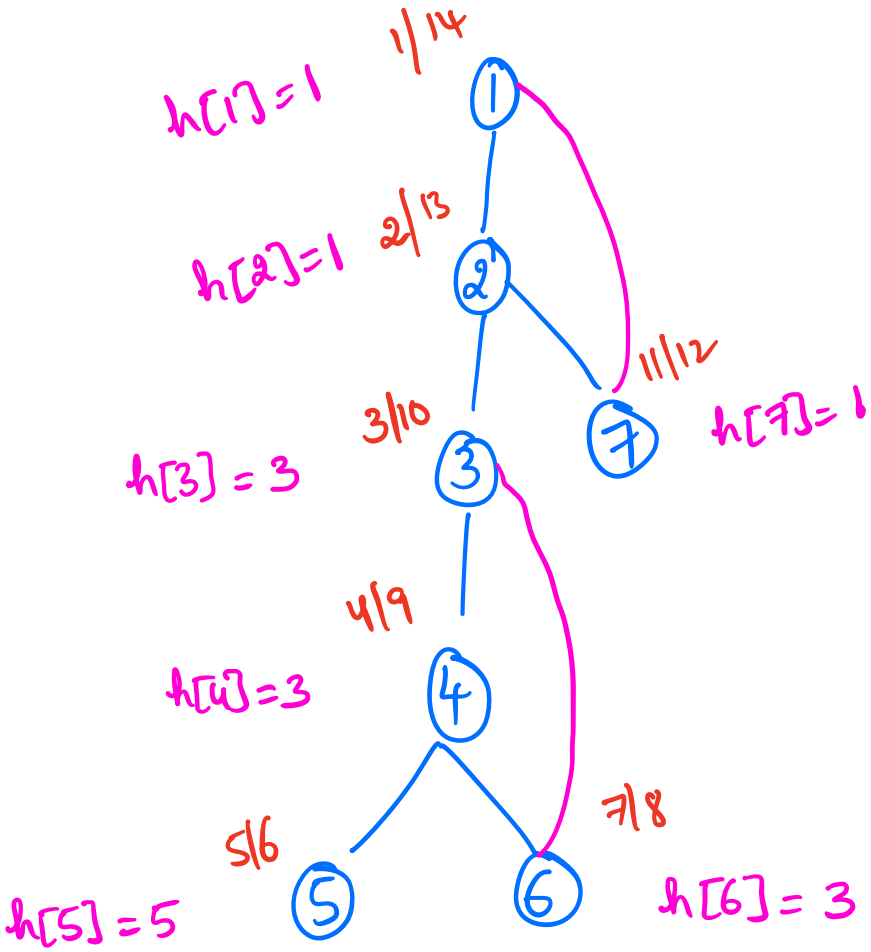
Pseudocode

Running time: $O(m+n)$

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)
```

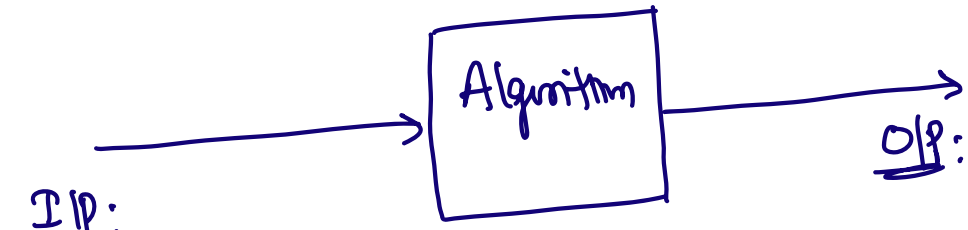
```
DFS-VISIT(G, u)
1  time = time + 1
2  u.d = time , h[u] = u.d        → Additional information
3  u.color = GRAY
4  for each v ∈ G.Adj[u]
5      if v.color == WHITE
6          v.π = u
7          DFS-VISIT(G, v)
8      h[u] = min{h[u], h[v]}
9      If h[v] > u.d
10         then uv is a cut edge      } v is not visited
11     else
12         h[u] = min{h[u], v.d}      } v is visited (uv is back edge)
13 u.color = black
14 time = time + 1
15 u.f = time
```





The tree edge from u to v is a bridge/cut-edge if $h[v] > u.d$

Algorithm to find Cut-vertices in a Graph



I/P:

Undirected graph G

+

Connected

O/P:

Cut vertices of G

Warmup:

① Can the root of the DFS tree be a cut vertex?

True | False :

The root of the Dfs tree is a cut vertex iff it has at least two children.

Easy Algorithm based on above True|False:

Run DFS once for each node.

Running time: $O(mn)$

m = # of edges

n = # of Vertices

Think about an idea which is

Similar to the cut edges algorithm?

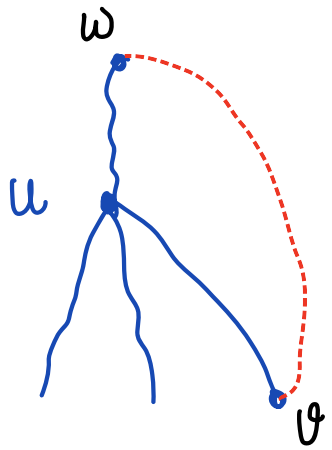
Define,

For each node u,

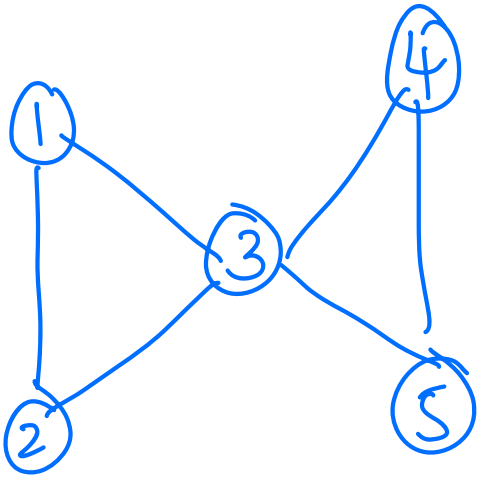
h[u] is the node with lowest discover time in the tree one can reach from u by

- going down zero or more tree edges
- going up on back edge, staying there if no back edge exists.

$$h[u] = \min \begin{cases} u.d, \\ w.d : (v,w) \text{ is a back edge} \\ \quad \text{for some descendant } v \text{ of } u \\ \quad \text{to some ancestor } w \text{ of } u \end{cases}$$



Example :



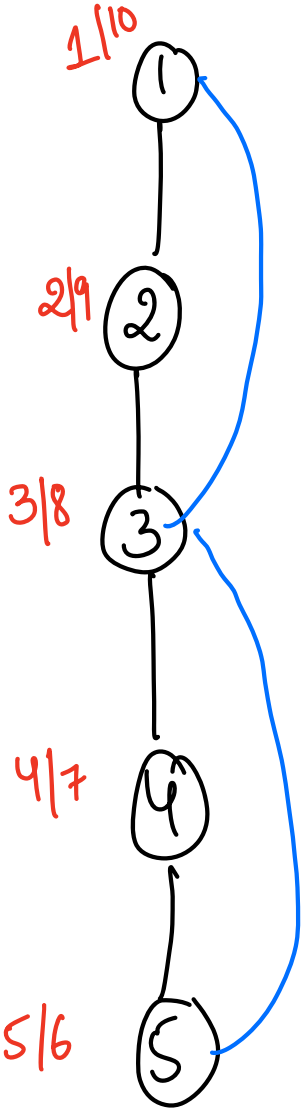
$h[1] = 1$

$h[2] = 1$

$h[3] = 1$

$h[4] = 3$

$h[5] = 3$

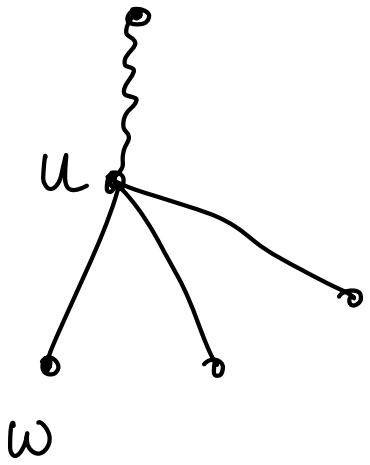


Q. Based on h values, how do we decide whether a vertex is cut-vertex

True/False: [u is NOT the root of the DFS tree]

u is a cut vertex if and only if

u has a child w with $h[w] \geq u.d$



Excercise : Write the Pseudo code